

Assignment 0

Task 1: In your terminal, run

In [1]: `conda info`

```

active environment : transformers
active env location : [REDACTED]\miniconda3\envs\transformers
shell level : 1
user config file : [REDACTED]\.condarc
populated config files :
conda version : 22.11.1
conda-build version : not installed
python version : 3.9.15.final.0
virtual packages : __archspec=1=x86_64
                  __cuda=13.4=0
                  __win=0=0
base environment : [REDACTED]\miniconda3 (writable)
conda av data dir : [REDACTED]\miniconda3\etc\conda
conda av metadata url : None
channel URLs : https://repo.anaconda.com/pkgs/main/win-64
               https://repo.anaconda.com/pkgs/main/noarch
               https://repo.anaconda.com/pkgs/r/win-64
               https://repo.anaconda.com/pkgs/r/noarch
               https://repo.anaconda.com/pkgs/msys2/win-64
               https://repo.anaconda.com/pkgs/msys2/noarch
package cache : [REDACTED]\miniconda3\pkgs
                [REDACTED]\.conda\pkgs
                [REDACTED]\AppData\Local\conda\conda\pkgs
envs directories : [REDACTED]\miniconda3\envs
                  [REDACTED]\.conda\envs
                  [REDACTED]\AppData\Local\conda\conda\envs
platform : [REDACTED]
user-agent : conda/22.11.1 requests/2.28.1 CPython/3.9.15 Windows/10 Wi
ndows/10.0.26200
administrator : False
netrc file : None
offline mode : False

```

Note: you may need to restart the kernel to use updated packages.

Task 2: Run all of Python commands in the table "Linear Algebra Equivalents" in Numpy for MATLAB Users.

In [2]: `import numpy as np`
`from scipy import linalg`
`from scipy.sparse.linalg import eigs, cg`

```
from scipy import signal
from IPython.display import display

a = np.array([
    [4.0, 1.0, 0.5, 0.2],
    [1.0, 3.0, 0.7, 0.3],
    [0.5, 0.7, 2.0, 0.4],
    [0.2, 0.3, 0.4, 1.0]
])

b = np.eye(4) * 2.0
b_vec = np.ones(4)

Z = np.random.rand(10, 4)
y = np.random.rand(10)
x_sig = np.arange(100.0)
q = 4

print("linalg.inv(a):")
display(linalg.inv(a))

print("linalg.pinv(a):")
display(linalg.pinv(a))

print("np.linalg.matrix_rank(a):")
display(np.linalg.matrix_rank(a))

print("linalg.solve(a, b):")
display(linalg.solve(a, b))

print("linalg.lstsq(a, b):")
display(linalg.lstsq(a, b))

print("x = linalg.solve(a.T, b.T).T:")
x = linalg.solve(a.T, b.T).T
display(x)

print("SVD: U, S, Vh = linalg.svd(a)")
U, S, Vh = linalg.svd(a)
print("U:"); display(U)
print("S:"); display(S)
print("Vh:"); display(Vh)
V = Vh.T
print("V = Vh.T:"); display(V)

print("linalg.cholesky(a):")
display(linalg.cholesky(a))

print("Eigen decomposition: D, V = linalg.eig(a)")
D, V = linalg.eig(a)
print("D:"); display(D)
print("V:"); display(V)

print("Generalized eigen: D, V = linalg.eig(a, b)")
D, V = linalg.eig(a, b)
```

```
print("D:"); display(D)
print("V:"); display(V)

print("eigs(a, k=2): D, V = eigs(a, k=2)")
D, V = eigs(a, k=2)
print("D:"); display(D)
print("V:"); display(V)

print("QR: Q, R = linalg.qr(a)")
Q, R = linalg.qr(a)
print("Q:"); display(Q)
print("R:"); display(R)

print("LU: P, L, U = linalg.lu(a)")
P, L, U = linalg.lu(a)
print("P:"); display(P)
print("L:"); display(L)
print("U:"); display(U)

print("Conjugate gradients: x_cg, info = cg(a, b_vec)")
x_cg, info = cg(a, b_vec)
print("x_cg:"); display(x_cg)
print("info:", info)

print("FFT: np.fft.fft(a)")
display(np.fft.fft(a))

print("IFFT: np.fft.ifft(a)")
display(np.fft.ifft(a))

print("np.sort(a):")
display(np.sort(a))

print("a_copy.sort(axis=0):")
a_copy = a.copy()
a_copy.sort(axis=0)
display(a_copy)

print("np.sort(a, axis=1):")
display(np.sort(a, axis=1))

print("a_copy.sort(axis=1):")
a_copy = a.copy()
a_copy.sort(axis=1)
display(a_copy)

print("Sort rows by first column:")
I = np.argsort(a[:, 0])
b_sorted = a[I, :]
print("I:"); display(I)
print("b_sorted:"); display(b_sorted)

print("Linear regression: x_lstsq = linalg.lstsq(Z, y)[0]")
x_lstsq = linalg.lstsq(Z, y)[0]
display(x_lstsq)
```

```

print("Downsample: signal.resample(x_sig, ceil(len(x_sig)/q))")
display(signal.resample(x_sig, int(np.ceil(len(x_sig) / q))))

print("np.unique(a):")
display(np.unique(a))

print("a.squeeze():")
display(a.squeeze())

```

```

linalg.inv(a):
array([[ 0.27586434, -0.08169279, -0.03721805, -0.01577781],
       [-0.08169279,  0.39087196, -0.10456237, -0.05909808],
       [-0.03721805, -0.10456237,  0.58493356, -0.1951611 ],
       [-0.01577781, -0.05909808, -0.1951611 ,  1.09894943]])
linalg.pinv(a):
array([[ 0.27586434, -0.08169279, -0.03721805, -0.01577781],
       [-0.08169279,  0.39087196, -0.10456237, -0.05909808],
       [-0.03721805, -0.10456237,  0.58493356, -0.1951611 ],
       [-0.01577781, -0.05909808, -0.1951611 ,  1.09894943]])
np.linalg.matrix_rank(a):
np.int64(4)
linalg.solve(a, b):
array([[ 0.55172869, -0.16338558, -0.07443609, -0.03155563],
       [-0.16338558,  0.78174392, -0.20912474, -0.11819616],
       [-0.07443609, -0.20912474,  1.16986713, -0.39032221],
       [-0.03155563, -0.11819616, -0.39032221,  2.19789886]])
linalg.lstsq(a, b):
(array([[ 0.55172869, -0.16338558, -0.07443609, -0.03155563],
       [-0.16338558,  0.78174392, -0.20912474, -0.11819616],
       [-0.07443609, -0.20912474,  1.16986713, -0.39032221],
       [-0.03155563, -0.11819616, -0.39032221,  2.19789886]]),
 array([], dtype=float64),
 np.int64(4),
 array([4.89092262, 2.55396085, 1.69699221, 0.85812432]))
x = linalg.solve(a.T, b.T).T:
array([[ 0.55172869, -0.16338558, -0.07443609, -0.03155563],
       [-0.16338558,  0.78174392, -0.20912474, -0.11819616],
       [-0.07443609, -0.20912474,  1.16986713, -0.39032221],
       [-0.03155563, -0.11819616, -0.39032221,  2.19789886]])
SVD: U, S, Vh = linalg.svd(a)
U:
array([[ -0.7867625 ,  0.6167563 , -0.0248097 , -0.00095749],
       [-0.53793495, -0.66548059,  0.5165881 , -0.02997183],
       [-0.28166966, -0.39267694, -0.81736283, -0.31366388],
       [-0.11087366, -0.15017347, -0.2538488 ,  0.94906045]])
S:
array([4.89092262, 2.55396085, 1.69699221, 0.85812432])
Vh:
array([[ -0.7867625 , -0.53793495, -0.28166966, -0.11087366],
       [ 0.6167563 , -0.66548059, -0.39267694, -0.15017347],
       [-0.0248097 ,  0.5165881 , -0.81736283, -0.2538488 ],
       [-0.00095749, -0.02997183, -0.31366388,  0.94906045]])
V = Vh.T:

```

```

array([[ -0.7867625 ,  0.6167563 , -0.0248097 , -0.00095749],
       [ -0.53793495, -0.66548059,  0.5165881 , -0.02997183],
       [ -0.28166966, -0.39267694, -0.81736283, -0.31366388],
       [ -0.11087366, -0.15017347, -0.2538488 ,  0.94906045]])
linalg.cholesky(a):
array([[2.          , 0.5          , 0.25          , 0.1          ],
       [0.          , 1.6583124   , 0.34673805, 0.15075567],
       [0.          , 0.          , 1.34806258, 0.23940081],
       [0.          , 0.          , 0.          , 0.95391823]])
Eigen decomposition: D, V = linalg.eig(a)
D:
array([4.89092262+0.j, 2.55396085+0.j, 1.69699221+0.j, 0.85812432+0.j])
V:
array([[ -0.7867625 , -0.6167563 , -0.0248097 , -0.00095749],
       [ -0.53793495,  0.66548059,  0.5165881 , -0.02997183],
       [ -0.28166966,  0.39267694, -0.81736283, -0.31366388],
       [ -0.11087366,  0.15017347, -0.2538488 ,  0.94906045]])
Generalized eigen: D, V = linalg.eig(a, b)
D:
array([2.44546131+0.j, 1.27698042+0.j, 0.84849611+0.j, 0.42906216+0.j])
V:
array([[ 0.7867625 , -0.6167563 ,  0.0248097 , -0.00095749],
       [ 0.53793495,  0.66548059, -0.5165881 , -0.02997183],
       [ 0.28166966,  0.39267694,  0.81736283, -0.31366388],
       [ 0.11087366,  0.15017347,  0.2538488 ,  0.94906045]])
eigs(a, k=2): D, V = eigs(a, k=2)
D:
array([4.89092262+0.j, 2.55396085+0.j])
V:
array([[ 0.7867625 +0.j,  0.6167563 +0.j],
       [ 0.53793495+0.j, -0.66548059+0.j],
       [ 0.28166966+0.j, -0.39267694+0.j],
       [ 0.11087366+0.j, -0.15017347+0.j]])
QR: Q, R = linalg.qr(a)
Q:
array([[ -0.96197214,  0.26250065,  0.07418748, -0.01411482],
       [ -0.24049304, -0.94500233,  0.21526391, -0.0528691 ],
       [ -0.12024652, -0.17850044, -0.96083108, -0.17459098],
       [ -0.04809861, -0.07875019, -0.15799149,  0.98311933]])
R:
array([[ -4.15812458, -1.78205339, -0.90906367, -0.36073955],
       [ 0.          , -2.72108172, -0.91875227, -0.38115094],
       [ 0.          , 0.          , -1.79708027, -0.46290725],
       [ 0.          , 0.          , 0.          , 0.89459925]])
LU: P, L, U = linalg.lu(a)
P:
array([[1., 0., 0., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 0., 0., 1.]])
L:
array([[1.          , 0.          , 0.          , 0.          ],
       [0.25         , 1.          , 0.          , 0.          ],
       [0.125        , 0.20909091, 1.          , 0.          ],
       [0.05         , 0.09090909, 0.17758879, 1.          ]])
U:

```

```

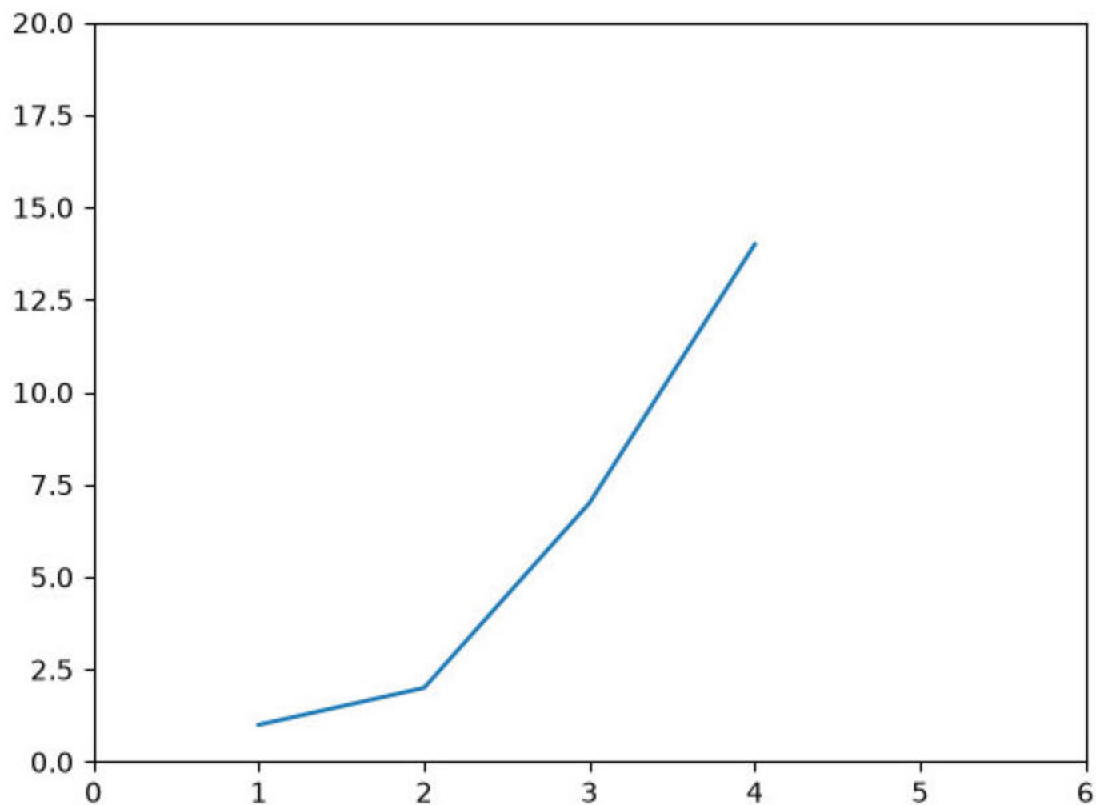
array([[4.          , 1.          , 0.5          , 0.2          ],
       [0.          , 2.75         , 0.575         , 0.25          ],
       [0.          , 0.          , 1.81727273, 0.32272727],
       [0.          , 0.          , 0.          , 0.90995998]])
Conjugate gradients: x_cg, info = cg(a, b_vec)
x_cg:
array([0.14117569, 0.14551872, 0.24799204, 0.82891243])
info: 0
FFT: np.fft.fft(a)
array([[ 5.7+0.j ,  3.5-0.8j,  3.3+0.j ,  3.5+0.8j],
       [ 5. +0.j ,  0.3-2.7j, -1.6+0.j ,  0.3+2.7j],
       [ 3.6+0.j , -1.5-0.3j,  1.4+0.j , -1.5+0.3j],
       [ 1.9+0.j , -0.2+0.7j, -0.7+0.j , -0.2-0.7j]])
IFFT: np.fft.ifft(a)
array([[ 1.425+0.j   ,  0.875+0.2j  ,  0.825+0.j   ,  0.875-0.2j  ],
       [ 1.25 +0.j   ,  0.075+0.675j, -0.4  +0.j   ,  0.075-0.675j],
       [ 0.9  +0.j   , -0.375+0.075j,  0.35 +0.j   , -0.375-0.075j],
       [ 0.475+0.j   , -0.05 -0.175j, -0.175+0.j   , -0.05 +0.175j]])
np.sort(a):
array([[0.2, 0.5, 1. , 4. ],
       [0.3, 0.7, 1. , 3. ],
       [0.4, 0.5, 0.7, 2. ],
       [0.2, 0.3, 0.4, 1. ]])
a_copy.sort(axis=0):
array([[0.2, 0.3, 0.4, 0.2],
       [0.5, 0.7, 0.5, 0.3],
       [1. , 1. , 0.7, 0.4],
       [4. , 3. , 2. , 1. ]])
np.sort(a, axis=1):
array([[0.2, 0.5, 1. , 4. ],
       [0.3, 0.7, 1. , 3. ],
       [0.4, 0.5, 0.7, 2. ],
       [0.2, 0.3, 0.4, 1. ]])
a_copy.sort(axis=1):
array([[0.2, 0.5, 1. , 4. ],
       [0.3, 0.7, 1. , 3. ],
       [0.4, 0.5, 0.7, 2. ],
       [0.2, 0.3, 0.4, 1. ]])
Sort rows by first column:
I:
array([3, 2, 1, 0])
b_sorted:
array([[0.2, 0.3, 0.4, 1. ],
       [0.5, 0.7, 2. , 0.4],
       [1. , 3. , 0.7, 0.3],
       [4. , 1. , 0.5, 0.2]])
Linear regression: x_lstsq = linalg.lstsq(Z, y)[0]
array([ 0.42432787,  0.47000973, -0.26127758,  0.08993698])
Downsample: signal.resample(x_sig, ceil(len(x_sig)/q))
array([ 37.5          , -4.45133654,  12.6470011 ,   8.78894489,
        18.4755782 ,  17.9621982 ,  25.75395463,  26.43948404,
        33.42539134,  34.66912547,  41.26666462,  42.77332801,
        49.20745587,  50.79254413,  57.22667199,  58.73333538,
        65.33087453,  66.57460866,  73.56051596,  74.24604537,
        82.0378018 ,  81.5244218 ,  91.21105511,  87.3529989 ,
        104.45133654])

```

```
np.unique(a):  
array([0.2, 0.3, 0.4, 0.5, 0.7, 1. , 2. , 3. , 4. ])  
a.squeeze():  
array([[4. , 1. , 0.5, 0.2],  
       [1. , 3. , 0.7, 0.3],  
       [0.5, 0.7, 2. , 0.4],  
       [0.2, 0.3, 0.4, 1. ]])
```

Task 3: Run the following script in IPython and paste the figure created by the script into your report.

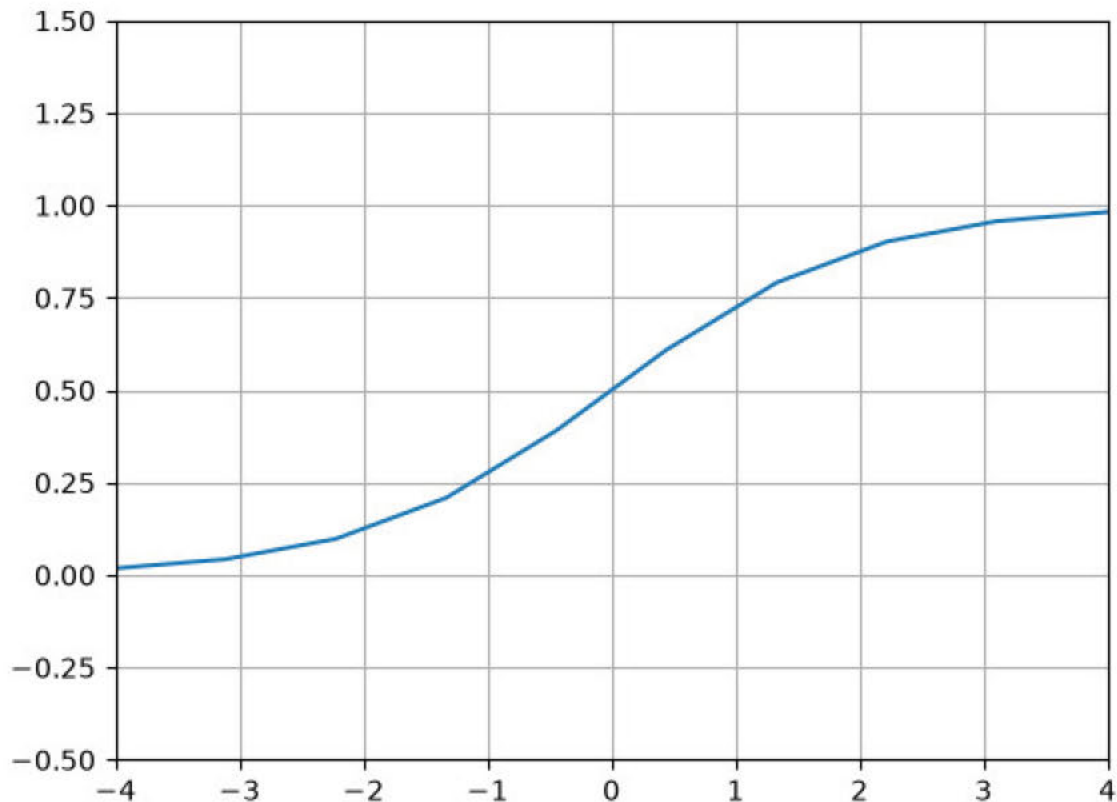
```
In [3]: import matplotlib.pyplot as plt  
plt.plot([1,2,3,4], [1,2,7,14])  
plt.axis([0, 6, 0, 20])  
plt.show()
```



Task 4: Use Matplotlib to create a figure of your choice in IPython. Paste your code and figure into your report.

```
In [4]: import numpy as np  
import matplotlib.pyplot as plt  
  
x = np.linspace(-4, 4, 10)  
y = 1 / (1 + np.exp(-x))
```

```
plt.plot(x, y)
plt.axis([-4, 4, -0.5, 1.5])
plt.grid()
plt.show()
```



Task 5: Paste your VCS account into your report.

Github

VCS account: SAMMYBOOOOM

Task 6: Start a new project in the IDE of your choice. Commit and push your project to GitHub as a public project. Paste the link of your project in your report.

Link: <https://github.com/SAMMYBOOOOM/Rice-COMP576>